1235. Maximum Profit in Job Scheduling Hard

Acceptance: 54.6% | LeetCode

We have `n` jobs, where every job is scheduled to be done from `startTime[i]` to `endTime[i]`, obtaining a profit of `profit[i]`.

You're given the `startTime`, `endTime` and `profit` arrays, return the maximum profit you can take such that there are no two jobs in the subset with overlapping time range.

If you choose a job that ends at time `X` you will be able to start another job that starts at time `X`.

Example 1:

```
**Input:** startTime = [1,2,3,3], endTime = [3,4,5,6],
profit = [50,10,40,70]
**Output:** 120
**Explanation:** The subset chosen is the first and
fourth job.
Time range [1-3]+[3-6] , we get profit of 120 = 50 +
70.
```

Example 2:

```
**Input:** startTime = [1,2,3,4,6], endTime =
[3,5,10,6,9], profit = [20,20,100,70,60]
**Output:** 150
**Explanation:** The subset chosen is the first,
fourth and fifth job.
Profit obtained 150 = 20 + 70 + 60.
```

Example 3:

```
**Input:** startTime = [1,1,1], endTime = [2,3,4],
profit = [5,6,4]
**Output:** 6
```

Constraints:

- `1 <= startTime.length == endTime.length == profit.length <= 5 * 104`
- `1 <= startTime[i] < endTime[i] <= 109`
- `1 <= profit[i] <= 104`

Claude Opus Solution

Maximum Profit in Job Scheduling

1. Intuition

The **key insight** is recognizing this as a variation of the **Weighted Interval Scheduling** problem—a classic dynamic programming pattern.

The core observations: - Unlike the basic "activity selection" problem (which is greedy), having **weights/profits** breaks the greedy approach. A job ending early might have low profit, while a longer job might be more valuable. - We must consider **every job**: either we **take it** (and skip all overlapping jobs) or we **skip it**. - If we **sort by end time**, then for any job we choose, we only need to find the latest non-overlapping job that finishes *before* this one starts. This creates an optimal substructure.

The recurrence: For each job, the answer is the max of: - **Including** this job: `profit + best_answer_before_this_job_starts` - **Excluding** this job: `best_answer_so_far`

To efficiently find "the best answer before this job starts", we use **binary search** since jobs are sorted.

2. Approach (Pseudocode)

```
1. Combine jobs as tuples (endTime, startTime, profit)
2. SORT jobs by endTime  (so we can binary search for compatible jobs)
3. dp[i] = max profit achievable using the first i jobs
   - Track parallel array of end_times for binary search
4. For each job i (1-indexed):
     a. Binary search: find latest job j whose endTime <= current job's startTime
     b. profit_with    = profit[i] + dp[j]   # take this job + best compatible
     c. profit_without = dp[i-1]             # skip this job
     d. dp[i] = max(profit_with, profit_without)
5. Return dp[n]
```

3. Python Solution

```
from typing import List
import bisect
```

```python
class Solution:
    def jobScheduling(self, startTime: List[int], endTime: List[int],
                      profit: List[int]) -> int:
        # ------------------------------------------------------------
        # STEP 1 & 2: Pair up jobs and sort by END time.
        # Sorting by end time lets us use binary search to quickly find
        # the latest job that finishes before the current one starts.
        # ------------------------------------------------------------
        jobs = sorted(zip(endTime, startTime, profit))  # (end, start, profit)

        n = len(jobs)

        # dp[i] = maximum profit considering the first i jobs (1-indexed).
        # dp[0] = 0 means "no jobs taken -> 0 profit".
        dp = [0] * (n + 1)

        # We need a sorted list of end times to binary search against.
        # Since 'jobs' is sorted by end time, this is already ordered.
        end_times = [job[0] for job in jobs]

        # ------------------------------------------------------------
        # STEP 4: Build up the dp table.
        # ------------------------------------------------------------
        for i in range(1, n + 1):
            end, start, prof = jobs[i - 1]  # current job (i-1 in 0-indexed jobs)

            # ----- Binary Search -----
            # Find the rightmost job whose end_time <= current job's start.
            # bisect_right gives the insertion point keeping list sorted;
            # it returns the count of jobs with end_time <= start.
            # That count IS the dp index of the best compatible prefix.
            #
            # Why bisect_right? A job ending exactly at 'start' is allowed
            # (problem states ending at X lets you start at X), so we include
            # equal end times -> use bisect_right.
            j = bisect.bisect_right(end_times, start)
            # 'j' is the number of jobs ending <= start, which maps to dp[j]
            # (since dp is 1-indexed over the same sorted job order).

            # ----- The Recurrence -----
            # Option A: TAKE this job -> its profit + best compatible (dp[j])
            take = prof + dp[j]

            # Option B: SKIP this job -> carry forward previous best (dp[i-1])
            skip = dp[i - 1]

            dp[i] = max(take, skip)

        # dp[n] = best profit using all n jobs available.
        return dp[n]
```

Why `bisect_right(end_times, start)` correctly maps to `dp[j]`

Because `dp` is 1-indexed and built in the **same sorted order** as `jobs`: - `dp[j]` = best profit using the first `j` jobs. - `bisect_right` returns how many jobs have `end_time <= start` — exactly the count of compatible earlier jobs, which equals the prefix length `j`.

---

4. Complexity Analysis

Aspect	Complexity	Explanation
Time	**O(n log n)**	Sorting takes `O(n log n)`. The main loop runs `n` times, and each iteration does a binary search in `O(log n)`. Total: `O(n log n) + O(n log n) = O(n log n)`.
Space	**O(n)**	The `dp` array, `jobs` list, and `end_times` list each use `O(n)` space.

---

5. Alternative Approaches

A) Top-Down Memoized Recursion (Reference Solution)

Sort by **start time** and recurse: at each job, either take it (jump to next compatible job via binary search) or skip it. Memoization avoids recomputation. Same `O(n log n)` complexity—just a recursive flavor of the same DP.

B) Heap-Based Sweep Line ⭐

A clean alternative that avoids explicit binary search:

```python
import heapq

class Solution:
    def jobScheduling(self, startTime, endTime, profit):
        # Sort by START time so we process jobs in chronological start order.
        jobs = sorted(zip(startTime, endTime, profit))

        # Min-heap of (endTime, max_profit_achievable_by_then)
        heap = []
        best = 0  # running max profit from all "finished" jobs

        for s, e, p in jobs:
            # Pop all jobs that END at or before current job's start.
            # Those jobs are now "available" to combine with current job.
            while heap and heap[0][0] <= s:
                best = max(best, heappop(heap)[1])

            # Push current job: best profit if we take it = best + p
            heappush(heap, (e, best + p))

        # Answer is the max profit across all entries still in heap + best.
        return max(best, max((p for _, p in heap), default=0))
```

This treats it as a **sweep line / event-based** problem. Also `O(n log n)`.

C) DP with Coordinate Compression

Compress timestamps to indices, then DP over time points. Useful when timestamps are bounded, but binary search approach is generally cleaner here.

Key Takeaways (Pattern Recognition)

1. **Weighted Interval Scheduling** → sort by end time + DP + binary search.
2. **Take-or-skip recurrence** is the heart of interval DP problems.
3. **Sorting + binary search** is a recurring trick to find "compatible predecessor" efficiently.
4. The "ends at X, can start at X" detail dictates `bisect_right` vs `bisect_left`—**always read overlap rules carefully**.

Codex Solution

1. Intuition

This is a classic **Weighted Interval Scheduling** problem.

Each job is an interval:

```
[startTime[i], endTime[i]]
```

with a weight/profit:

```
profit[i]
```

We want to choose non-overlapping intervals with maximum total profit.

The key insight is:

> For every job, we have two choices:
>
> 1. Take this job, then jump to the next job that starts after or exactly when this job ends.
> 2. Skip this job and consider the next one.

Because jobs can be sorted by start time, we can use **binary search** to quickly find the next compatible job.

This gives us a Dynamic Programming + Binary Search solution.

2. Approach

Step-by-step pseudocode

```
Combine startTime, endTime, and profit into jobs
Sort jobs by start time

Create an array starts containing only the sorted start times

Let dp[i] represent:
    maximum profit we can earn considering jobs from index i to the end

Initialize dp[n] = 0

Iterate i from n - 1 down to 0:

    current_start = jobs[i].start
    current_end = jobs[i].end
    current_profit = jobs[i].profit

    Find next_index using binary search:
        first job whose start time >= current_end

    profit_if_take = current_profit + dp[next_index]

    profit_if_skip = dp[i + 1]

    dp[i] = max(profit_if_take, profit_if_skip)

Return dp[0]
```

3. Python Solution

```python
from typing import List
from bisect import bisect_left


class Solution:
    def jobScheduling(
        self,
        startTime: List[int],
        endTime: List[int],
        profit: List[int]
    ) -> int:
        n = len(startTime)

        # Combine the job information into tuples:
        # Each job is represented as (start, end, profit)
        jobs = list(zip(startTime, endTime, profit))

        # Sort jobs by start time.
        # This allows us to binary search for the next non-overlapping job.
        jobs.sort()

        # Extract only the start times from sorted jobs.
        # We will binary search on this list.
        starts = [job[0] for job in jobs]

        # dp[i] = maximum profit obtainable starting from job index i
        #
        # dp[n] = 0 means:
        # If there are no jobs left to consider, profit is 0.
        dp = [0] * (n + 1)

        # Process jobs from right to left.
        # This works because dp[i] depends on future states:
```

```
        # dp[i + 1] and dp[next_index]
        for i in range(n - 1, -1, -1):
            current_start, current_end, current_profit = jobs[i]

            # Find the first job whose start time is >= current_end.
            #
            # This is valid because if one job ends at time X,
            # another job starting at time X is allowed.
            next_index = bisect_left(starts, current_end)

            # Option 1:
            # Take the current job.
            # Then we can only take jobs starting from next_index onward.
            take_profit = current_profit + dp[next_index]

            # Option 2:
            # Skip the current job.
            # Then the best profit is whatever we can get from i + 1 onward.
            skip_profit = dp[i + 1]

            # Choose the better option.
            dp[i] = max(take_profit, skip_profit)

        # dp[0] represents the maximum profit considering all jobs.
        return dp[0]
```

---

4. Complexity Analysis

Let `n` be the number of jobs.

Time Complexity

```
O(n log n)
```

Explanation:

- Sorting the jobs takes:

```
O(n log n)
```

- For each job, we perform one binary search:

```
O(log n)
```

- There are `n` jobs, so all binary searches cost:

```
O(n log n)
```

Total:

```
O(n log n)
```

Space Complexity

```
O(n)
```

Explanation:

We store:

- `jobs`, which has `n` elements
- `starts`, which has `n` elements
- `dp`, which has `n + 1` elements

So the total extra space is:

```
O(n)
```

---

5. Alternative Approaches

1. Top-down DFS + Memoization

Instead of filling `dp` bottom-up, we can define:

```
dfs(i) = max profit starting from job i
```

Then:

```
dfs(i) = max(
    profit[i] + dfs(next compatible job),
    dfs(i + 1)
)
```

This is essentially the same recurrence, but implemented recursively.

Time complexity:

```
O(n log n)
```

Space complexity:

```
O(n)
```

---

2. Sort by End Time

Another common weighted interval scheduling approach is:

- Sort jobs by end time.
- Let `dp[i]` represent the best profit using jobs up to index `i`.
- For each job, binary search for the latest job that ends before this job starts.

The recurrence becomes:

```
dp[i] = max(
    dp[i - 1],
    profit[i] + dp[previous_non_overlapping_job]
)
```

This also runs in:

```
O(n log n)
```

---

3. Min-Heap Sweep Line

We can sort jobs by start time and use a min-heap based on end time.

The heap stores pairs:

```
(end_time, accumulated_profit)
```

As we process jobs in increasing start time:

- Pop all jobs that ended before the current job starts.
- Update the best profit so far.
- Push the current job with `best_profit + current_profit`.

This also gives:

```
O(n log n)
```

but the DP + binary search version is usually easier to reason about.